

ATLASky-AI: An Adaptive Multi-Agent Framework for Autonomous 4D Knowledge Verification in Aerospace

RAED AWILL

Abstract

Accurate integration of heterogeneous spatiotemporal data into knowledge graphs is paramount for ensuring safety and efficiency in aerospace operations. While recent research has advanced knowledge graph construction through large language models (LLMs), verification of dynamic spatiotemporal data remains largely overlooked. To fill this gap, we propose ATLASky-AI, a novel agent-based framework aimed at autonomously constructing and verifying four-dimensional spatiotemporal knowledge graphs (4D STKG). The general idea of ATLASky-AI is to combine large language models with an innovative verification architecture-TruthFlow-that systematically validates data before integration. First, we propose the Ranked Multi-Modal Verification (RMMVe) process, which orchestrates five specialized verification modules in a computationally efficient sequence with early termination capabilities. Subsequently, we design the Autonomous Adaptive Intelligence Cycle (AAIC) that dynamically refines verification parameters using CGR-CUSUM monitoring. With ATLASky-AI, we present two complementary verification mechanisms: the cost-effective RMMVe process and the adaptive AAIC feedback loop, each imposing different verification constraints. Our experimental evaluations across multiple aerospace datasets demonstrate verification success rates ranging from 70% to 93%, significantly outperforming conventional verification approaches. Furthermore, we find that our case study in aerospace company AddQual illustrates how ATLASky-AI enhances decision-making, reduces knowledge graph creation error rates from 15% to 2.5%, and lowers operational costs.

Keywords

adaptive intelligence, aerospace, components, circuits, devices and systems, computing and processing, engineering profession, knowledge graph verification, multi-agent systems, spatiotemporal knowledge graphs, trustworthy ai

ATLASky-AI: An Adaptive Multi-Agent Framework for Autonomous 4D Knowledge Verification in Aerospace

Raed Awill [✉], Wajahat Ali Khan [✉], Maqbool Hussain [✉], Ben Anderson [✉], S. M. Ahsan Kazmi [✉]

Abstract—Accurate integration of heterogeneous spatiotemporal data into knowledge graphs is paramount for ensuring safety and efficiency in aerospace operations. While recent research has advanced knowledge graph construction through large language models (LLMs), verification of dynamic spatiotemporal data remains largely overlooked. To fill this gap, we propose ATLASky-AI, a novel agent-based framework aimed at autonomously constructing and verifying four-dimensional spatiotemporal knowledge graphs (4D STKG). The general idea of ATLASky-AI is to combine large language models with an innovative verification architecture—TruthFlow—that systematically validates data before integration. First, we propose the Ranked Multi-Modal Verification (RMMVe) process, which orchestrates five specialized verification modules in a computationally efficient sequence with early termination capabilities. Subsequently, we design the Autonomous Adaptive Intelligence Cycle (AAIC) that dynamically refines verification parameters using CGR-CUSUM monitoring. With ATLASky-AI, we present two complementary verification mechanisms: the cost-effective RMMVe process and the adaptive AAIC feedback loop, each imposing different verification constraints. Our experimental evaluations across multiple aerospace datasets demonstrate verification success rates ranging from 70% to 93%, significantly outperforming conventional verification approaches. Furthermore, we find that our case study in aerospace company AddQual illustrates how ATLASky-AI enhances decision-making, reduces knowledge graph creation error rates from 15% to 2.5%, and lowers operational costs.

Index Terms—Spatiotemporal Knowledge Graphs, Knowledge Graph Verification, Multi-Agent Systems, Adaptive Intelligence, Trustworthy AI.

I. INTRODUCTION

SPATIOTEMPORAL data management is increasingly vital in aerospace maintenance [1], [2], [3], [4], where data accuracy impacts safety and operational efficiency. Studies show that 48% of maintenance incidents occur due to skill-based errors [5], costing the aerospace industry over \$1 billion annually [6]. Moreover, the aerospace sector faces significant challenges due to strict safety standards and the complexity of its data environments [7], [8]. One of the key challenges in aerospace maintenance, especially for critical components like jet engines (see Figure 1), is the integration and verification of spatiotemporal data from diverse sources like maintenance logs, sensor readings, and 3D spatial configurations [9]. Engineers are often tasked with manually collecting and verifying data, a process that is inherently labor-intensive and sometimes subject to human error [10]. This traditional approach not only slows down operations but also incurs substantial financial and time-related costs [10]. Automated verification, such as

ATLASky-AI, mitigates these challenges through improved accuracy, reduced human intervention, and faster decision-making.

In addition, Knowledge Graphs (KGs) offer a structured means of representing entities, relationships, and attributes extracted from heterogeneous data sources, thereby resolving data inconsistencies and enabling robust, scalable integration. Unlike traditional relational databases, KGs facilitate the flexible and dynamic integration of multimodal data [11]. Their graph-based structure effectively connects technical documentation, sensor readings, maintenance logs, visual inspections, and historical data [11]. However, existing KG frameworks often assume data correctness without rigorously verifying the reliability of information—a critical limitation in safety-critical domains.

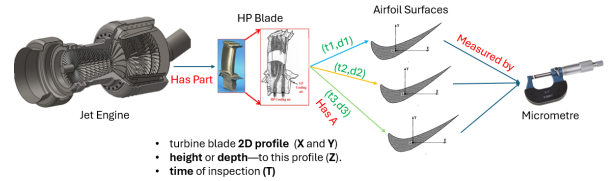


Fig. 1: Representation of 4D data for turbine blade maintenance, illustrating the turbine blade’s 2D profile (X, Y), height/depth (Z), and time of inspection (T).

Recent advances in large language models (LLMs) such as GPT-3 [12] and GPT-4 [13] demonstrate AI’s ability to navigate complex datasets and extract knowledge. However, these models can generate inaccurate but seemingly credible information (hallucinations) [14]. In aerospace, such inaccuracies risk operational errors [15]. For example, an LLM might misinterpret sensor data, leading to incorrect assessments of component conditions and maintenance failures. Existing verification approaches often handle spatial or temporal constraints independently, lacking a comprehensive “4D” verification framework capable of concurrently addressing spatial consistency, temporal continuity, and domain-specific semantics. This challenge, coupled with the complexity of aerospace systems, raises key research questions:

- How can we develop an autonomous system capable of accurately extracting, verifying, and integrating spatiotemporal data while minimizing errors?

- *How can we ensure the reliability of AI-generated insights in safety-critical aerospace applications?*
- *How can we balance verification accuracy and computational efficiency in real-time operations?*

To address these questions, we propose ATLASKy-AI, an AI agent framework for constructing and verifying 4D spatiotemporal knowledge graphs. Unlike previous approaches that focus primarily on knowledge representation or reasoning with an assumption of data veracity, our framework takes a verification-first approach, autonomously extracting and validating data across spatial, temporal, and semantic contexts before integration into a structured graph. This approach enhances aerospace safety and efficiency and addresses common challenges across other safety-critical industries [16].

Our key contributions include:

- A novel framework for automated verification of 4D spatiotemporal knowledge graphs, achieving high verification success rates across diverse aerospace datasets with significant improvements in both accuracy and verification time.
- The introduction of the TruthFlow architecture, which implements Ranked Multi-Modal Verification (RMMVe) within a multi-agent system. This design features a unique Shapley-value integration of verification agents and early stopping mechanisms to optimize efficiency.
- An Autonomous Adaptive Intelligence Cycle (AAIC), which dynamically adjusts verification parameters based on real-time performance metrics, ensuring high accuracy and computational efficiency in dynamic data environments.

II. RELATED WORK

Knowledge graphs (KGs) have become essential for representing and integrating structured information across diverse domains. Early projects like Freebase [17], DBpedia [18], and YAGO [19] laid the foundation by using semi-automated extraction from structured sources. More recently, LLMs such as GPT-3 [12] and GPT-4 [13] have accelerated KG construction by automating entity and relation extraction. Despite this progress, challenges remain, particularly in handling spatiotemporal relationships [20] and mitigating hallucinations (i.e., fabricated or inaccurate information) [21]. A critical issue in KG verification is ensuring the accuracy of extracted facts, especially with dynamic data. Frameworks like KG-Agent [22] and World Avatar [23] address static and real-time reasoning but lack comprehensive spatiotemporal validation. KGValidator [24] offers autonomous triple validation but provides only basic temporal or spatial support. Hallucination rates in LLM-generated content can reach up to 25%, posing risks in critical applications [21]. To reduce hallucinations, systems like Retrieval-Augmented Generation (RAG) [25] and RETRO [26] ground outputs in verified sources, while SelfCheckGPT [27] and MetaCheckGPT [28] use uncertainty modeling. However, these methods do not fully address dynamic or domain-specific constraints. Embedding-based veri-

fication [29] captures semantic relationships but struggles with rapidly changing data. Multi-agent systems enhance adaptability through reactive and deliberative agents [30]. Hybrid architectures [31] and multi-agent reinforcement learning [32] improve scalability, but many systems lack real-time monitoring or spatiotemporal precision. Multi-modal and spatiotemporal validation techniques [33] are pivotal for enhancing knowledge graph quality by integrating diverse data sources. However, these techniques often encounter significant computational constraints that limit their scalability. To address this, early termination methods [34] have been proposed, which reduce computational overhead by ceasing further verification once predefined confidence thresholds are reached. Despite this efficiency gain, such approaches rarely incorporate adaptive logic that could dynamically adjust verification parameters based on real-time data characteristics. Furthermore, although specialized modules exist for handling spatial or temporal validation independently [35], a comprehensive “4D” verification framework that is capable of concurrently addressing spatial and temporal dimensions remains unexplored. Our proposed ATLASKy-AI framework (see Figure 3) addresses these gaps by integrating the Ranked Multi-Modal Verification (RMMVe) process with an Autonomous Adaptive Intelligence Cycle (AAIC). This design unifies multi-agent collaboration, real-time performance monitoring via CGR-CUSUM [36], and specialized spatiotemporal modules, ensuring rigorous validation. By dynamically refining strategies and using early termination, ATLASKy-AI achieves computational efficiency without compromising accuracy, offering a robust, scalable approach to knowledge graph verification in dynamic domains.

III. PRELIMINARIES

This section establishes the formal foundations of the Spatiotemporal Knowledge Graphs and introduces the multi-modal verification principles essential to our framework.

A. Four-Dimensional Spatiotemporal Knowledge Graph Representation (4D STKG)

A 4D STKG extends traditional knowledge graphs by incorporating three spatial dimensions (X, Y, Z) and a temporal dimension T , enabling the representation of dynamic relationships over time [37]. This approach is particularly crucial for aerospace operations, where safety and efficiency depend on accurate spatiotemporal data integration [38].

Constructed using a custom ontology $\mathcal{O}$, the 4D STKG integrates heterogeneous data sources while maintaining consistency and flexibility. The ontology $\mathcal{O} = (\mathcal{E}_o, \mathcal{R}_o, \mathcal{A}_o, \mathcal{S}_o)$ defines entity classes $\mathcal{E}_o$, relationship types $\mathcal{R}_o$, attribute types $\mathcal{A}_o$, and spatiotemporal properties $\mathcal{S}_o$. The set of vertices $\mathcal{V} = \{v_1, v_2, \dots, v_P\}$ represents time-aware entities in the graph (see Figure 2). Each vertex $v_i = (e_i, a_i, s_i)$ is characterized by its entity class $e_i \in \mathcal{E}_o$, attributes $a_i \in \mathcal{A}_o$, and spatiotemporal properties $s_i = (X_i, Y_i, Z_i, T_i) \in \mathbb{R}^4$. The vertex set at any specific time $\mathcal{V}(t)$ includes all vertices that are active at time t , allowing the graph to capture the dynamic nature of entities (More details in Supplementary Appendix B)

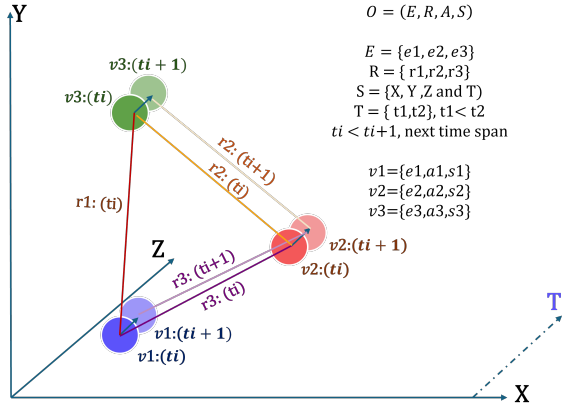


Fig. 2: Illustration of a 4D STKG integrated with its underlying ontology $\mathcal{O}$. Vertices v_1 , v_2 , and v_3 are positioned in a three-dimensional space (X, Y, Z) and evolve over discrete time points $(t_i$ and $t_{i+1})$, along the temporal dimension T . Each vertex is annotated with its corresponding entity class, attributes, and spatiotemporal properties as defined in $\mathcal{O}$. Relationships r_1 , r_2 , and r_3 illustrate the evolving connections among these vertices, thereby encapsulating both the conceptual schema and the dynamic instantiation within the KG.

B. Problem Formulation

Consider a 4 Dimensional STKG $\mathcal{G} = (V, E, \Sigma)$, where V is the set of vertices, E is the set of relations, and Σ embeds the spatiotemporal attributes (X, Y, Z, T) . The central challenge addressed in this paper is how to accurately verify and integrate new knowledge from heterogeneous sources into the existing 4D STKG while maintaining data integrity in safety-critical aerospace applications, with an emphasis on computational efficiency.

For each incoming data instance $d_k \in \mathcal{D}$ originating from heterogeneous sources, a candidate fact $\{e_k, r_k, a_k, s_k\}$ is extracted using an LLM-based pipeline. These candidate facts must undergo rigorous verification before integration into the knowledge graph, as LLMs may generate plausible but incorrect information that could compromise safety in aerospace contexts. The verification process is designed to balance thoroughness with computational efficiency by employing a ranked, sequential approach that can terminate early when sufficient confidence is achieved.

Let the set of verification modules be $\mathcal{M} = \{M_1, M_2, M_3, M_4, M_5\}$, representing Local Ontology (LOV), Public Ontology (POV), Multi-Agent (MAV), Web Search (WSV), and Embedding Similarity (ESV) verification methods respectively. Each module M_i yields a confidence score $C_{\text{module}_i}(d_k)$ (Eq. 2) and the aggregated confidence is $C_{\text{total}}(d_k)$ (Eq. 3). The decision function $\delta(d_k)$ that determines whether to accept a data point is defined as:

$$\delta(d_k) = \begin{cases} 1, & \text{if } C_{\text{total}}(d_k) \geq \Theta \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

where Θ is the global confidence threshold. When $\delta(d) = 1$, the extracted fact is integrated into $\mathcal{G}$; otherwise, it is rejected

or flagged for manual review.

Additionally, our framework continuously monitors the performance $p_i(t)$ of each verification module and dynamically updates parameters to optimize verification accuracy and efficiency. Table I summarizes the key notations used throughout this paper.

TABLE I: Notations and Descriptions

Notation	Description
Θ	Overall confidence threshold for knowledge integration
$M_1, M_2, \dots, M_5$	Verification modules (LOV, POV, MAV, WSV, ESV)
C_{module_i}	Confidence score for verification module M_i
w_i	Weight assigned to verification module M_i
P_{LOV}, R_{LOV}	Precision and Recall for LOV
A_{POV}, C_{APOV}	Accuracy and Coverage for POV
CS_{MAV}, R_{MAV}	Consensus score and reliability for MAV
$R_{WSV}, F1_{WSV}$	Recall and Precision for WSV
Sim_{ESV}, AD_{ESV}	Similarity score and anomaly detection rate for ESV
α_i	Weighting factor for each confidence score calculation
$p_i(t)$	Performance metric for M_i at time t
$S_i(t)$	Cumulative sum of deviations from target performance for module M_i
h	Predefined threshold for detecting significant deviations in performance

C. Ethical and Safety Considerations

The framework mitigates LLM hallucination risks using multi-tiered verification thresholds and cross-validation techniques within the RMMVe process. This approach ensures compliance with critical standards, including ISO/IEC 42001:2023 [39] for establishing an AI Management System and ISO/IEC 27001:2022 [40] for robust information security management. A synthetic dataset is used for AddQual during testing to protect sensitive information.

IV. ATLASKY-AI FRAMEWORK: APPROACH

In this section, we present the proposed *ATLASky-AI* framework for reliable, adaptive knowledge representation. Our methodology comprises three main pools: (i) *Data Acquisition and Preprocessing*, (ii) *Knowledge Extraction*, and (iii) *Truth-Flow Verification and Knowledge Integration*. These pools are illustrated in Figure 3, which shows the end-to-end workflow from raw data ingestion to knowledge graph integration, along with autonomous adaptiveness for real-time optimization.

A. Data Acquisition and Preprocessing

At the initial stage of the pipeline (Figure 3), raw multimodal data, denoted $RD = \{rd_1, rd_2, \dots, rd_n\}$, are uploaded and pass through basic preprocessing (e.g., cleaning, normalization). The goal is to transform these heterogeneous inputs into a form compatible with subsequent LLM prompts and domain-specific analysis. Typical preprocessing steps may include: (1.) *Textual Filtering*: Removing odd tokens, stopwords, or non-relevant metadata. (2.) *Structural Alignment*: Converting tabular, sensor, or image-derived metadata into standardized formats. (3.) *Time & Coordinate Normalization*: Aligning timestamps or spatial coordinates to ensure consistency. By the end of this pool, the pipeline produces a cleaner set of input data, ready for LLM-based knowledge extraction.

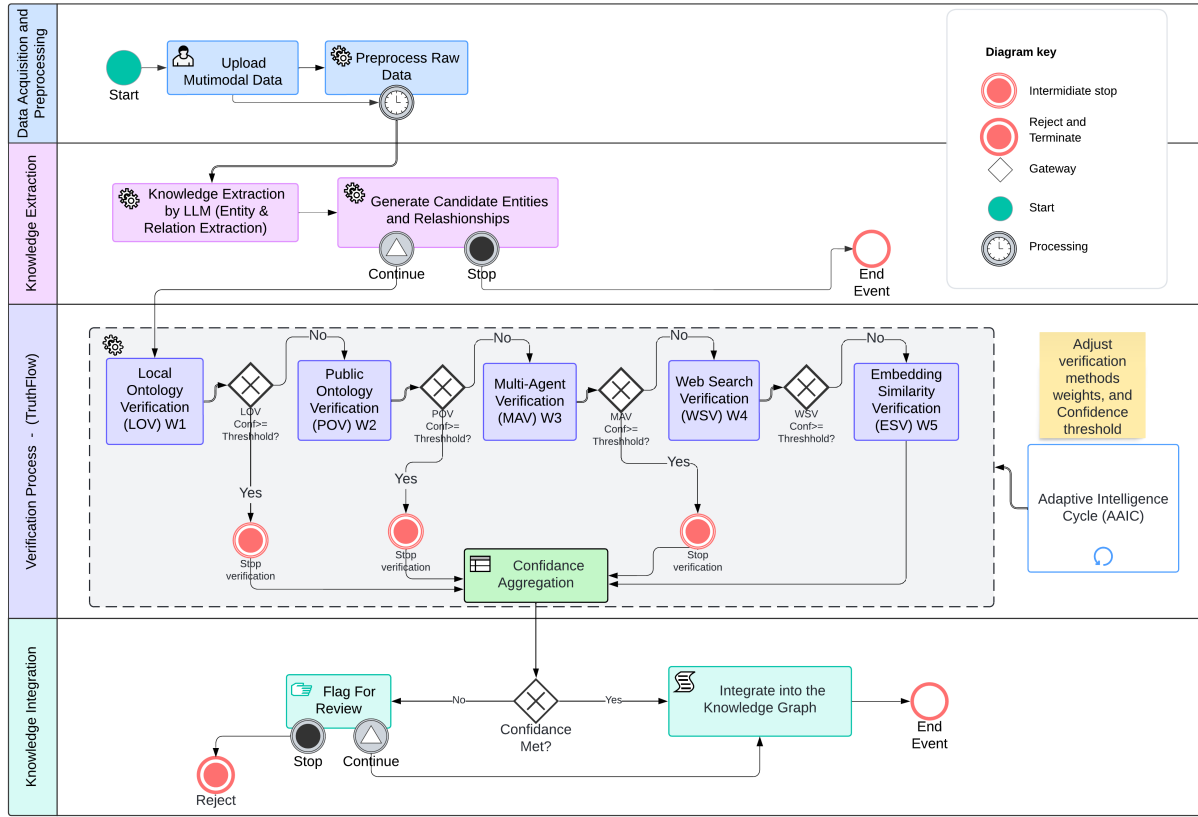


Fig. 3: BPMN diagram of the ATLASKy-AI framework: From data acquisition and LLM-based extraction to TruthFlow verification (LOV, POV, MAV, WSV, ESV), confidence aggregation, and final integration into the 4D STKG. The AAIC dynamically updates verification parameters (weights and thresholds) based on real-time feedback.

B. Knowledge Extraction

The preprocessed data flows into the *Knowledge Extraction* pool (represented by “Knowledge Extraction by LLM” and “Generate Candidate Entities & Relationships” in Figure 3). Here, an LLM-based pipeline employs domain-specific prompts enriched with aerospace terminology, maintenance protocols, and regulatory standards and then generates candidate knowledge (More details in supplementary Appendix A). These prompts incorporate spatial reference disambiguation (e.g., clarifying coordinate systems) and temporal context specification (e.g., maintenance cycle phases). Formally, this process is modeled as: $\phi_{LLM} : RD \xrightarrow{\text{prompts}} \mathcal{D}$, where $\mathcal{D} = \{d_1, d_2, \dots, d_k\}$ and each candidate d_k is a tuple: $d_k = (e_k, r_k, a_k, s_k)$. Here, e_k denotes entities (e.g., turbine blades, inspection tools), r_k represents relationships (e.g., *is_part_of*, *has_tolerance*), a_k captures attributes (e.g., material composition), and $s_k = (X_k, Y_k, Z_k, T_k)$ contains the spatiotemporal coordinates critical for maintenance timeline tracking and component positioning. The extracted candidate knowledge $\mathcal{D}$ is then forwarded to TruthFlow process for rigorous validation and subsequent integration into 4D STKG.

C. Verification Process: TruthFlow

Figure 3 illustrates how our proposed TruthFlow orchestrates the verification workflow. This system comprises two comple-

mentary stages:

- 1) **Ranked Multi-Modal Verification Process (RMMVe)**: A sequential, multi-step procedure that applies specialized modules to verify data across textual, numerical, temporal, and semantic dimensions.
- 2) **Autonomous Adaptive Intelligence Cycle (AAIC)**: A real-time feedback loop that adjusts the verification module parameters based on performance metrics collected during the RMMVe.

In essence, the RMMVe processes each data instance sequentially through its specialized modules before being integrated into $\mathcal{G}$. Meanwhile, the AAIC continuously monitors module performance and dynamically fine-tunes RMMVe parameters to ensure robust and accurate verification under evolving data conditions.

The following subsections detail each component:

1) *Ranked Multi-Modal Verification Process*: RMMVe is a foundational element of our framework, designed to ensure the reliability of data extracted from multiple modalities before integration into the knowledge graph. By verifying data through a sequence of specialized modules (textual, numerical, temporal, semantic), the RMMVe mitigates AI hallucinations and inaccurate extractions. It is adaptable to a variety of safety-critical domains since each verification module can be

tailored with domain-specific ontologies or specialized agents. For example, in aerospace, POV could leverage iof-maint ontology [41], while MAV might focus on physiological data consistency. (The rationale of each verification module in the supplementary Appendix D)

a) Mathematical Framework and Algorithm: Formally, as $\mathcal{D} = \{d_1, d_2, \dots, d_k\}$ be the set of candidate data elements extracted by the module (cf. Section IV-B). Our goal is to determine whether each d_k should be integrated into the knowledge graph $\mathcal{G}$. We define a set of verification modules $\mathcal{M}$. Each module M_i computes a confidence score $C_{\text{module}_i}(d_k) \in [0, 1]$. The modules are executed in a ranked order to prioritize cost-effective and domain-trusted verifications first. Algorithm 1 states that if $C_{\text{module}_i}(d_k)$ meets or exceeds its threshold θ_i , the system proceeds directly to confidence aggregation (thus saving resources); otherwise, the next module is activated. In this way, the verification process balances computational efficiency with rigorous checks (Operational scenario example in the supplementary Appendix E).

Module Score Computation: Let $C_{\text{module}_i}(d_k)$ be:

$$C_{\text{module}_i}(d_k) = w_i \left(\alpha_i \cdot \text{Metric}_1^{(i)}(d_k) + (1 - \alpha_i) \text{Metric}_2^{(i)}(d_k) \right), \quad (2)$$

where $w_i \in [0, 1]$ is the dynamic weight assigned to module M_i , $\alpha_i \in [0, 1]$ balances two primary metrics, and $\text{Metric}_1^{(i)}, \text{Metric}_2^{(i)}$ are module-specific performance measures (e.g., precision, recall). This formulation is domain-agnostic, so each module can adjust its metrics according to the application domain.

Overall Confidence: The aggregated confidence $C_{\text{total}}(d_k)$ is computed as the arithmetic mean of the confidence scores from the activated verification modules. Let $\mathcal{A}$ denote the set of activated modules for d_k ; then,

$$C_{\text{total}}(d_k) = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} C_{\text{module}_i}(d_k). \quad (3)$$

As stated in Eq. 1, if $C_{\text{total}}(d_k) \geq \Theta$, the data d_k is accepted into the knowledge graph $\mathcal{G}$; otherwise, it is rejected or flagged for review. Over time, the set of activated modules $\mathcal{A}$ and their parameters (e.g., thresholds θ_i) are dynamically updated by the AAIC IV-C2.

b) Modules Formalism: Below we give more details of how each module's score is computed.

(I) Local Ontology Verification (LOV): Here we use the predefined custom local ontology $\mathcal{O}$ (III-A). Let $E_{\text{extracted}}$ and $R_{\text{extracted}}$ denote the sets of entities and relationships extracted from data d_k . The sets of correctly identified entities and relationships are: $E_{\text{correct}} = E_{\text{extracted}} \cap \mathcal{E}_{\text{local}}$ and $R_{\text{correct}} = R_{\text{extracted}} \cap \mathcal{R}_{\text{local}}$. Define precision P_{LOV} and recall R_{LOV} as: $P_{\text{LOV}} = \frac{|E_{\text{correct}}| + |R_{\text{correct}}|}{|E_{\text{extracted}}| + |R_{\text{extracted}}|}$, and $R_{\text{LOV}} = \frac{|E_{\text{correct}}| + |R_{\text{correct}}|}{|\mathcal{E}_{\text{relevant}}| + |\mathcal{R}_{\text{relevant}}|}$, where $\mathcal{E}_{\text{relevant}}$ and $\mathcal{R}_{\text{relevant}}$ are the sets of all relevant entities and relationships in the ontology for the given context. The confidence score is:

$$C_{\text{LOV}}(d_k) = w_1 \left(\alpha_1 P_{\text{LOV}} + (1 - \alpha_1) R_{\text{LOV}} \right).$$

Algorithm 1: RMMVe within the TruthFlow Architecture

Input: Extracted data d_k

Output: Decision to accept, reject, or flag d_k for manual review

Initialize: For each verification module M_i ($i = 1, \dots, n$), set global threshold θ , module threshold θ_i , weight w_i , and weighting factor α_i ;

for each module M_i in the predefined order do
 1) Compute Module Confidence: Calculate $C_{\text{module}_i}(d_k)$ using Eq. (2);
 if $C_{\text{module}_i}(d_k) \geq \theta_i$ **then**
 ; // Sufficient confidence achieved; terminate early
 break;
 end
 else
 ; // Insufficient confidence; proceed to next module
 continue;
 end
end

Overall Confidence: Compute $C_{\text{total}}(d_k)$ using Eq. (3);

if $C_{\text{total}}(d_k) \geq \Theta$ **then**
 Accept d_k into $\mathcal{G}$;
end
else
 Reject d_k or flag for manual review;
end

Update: Update parameters w_i , θ_i , and α_i using AAIC (see Subsection IV-C2);

If $C_{\text{LOV}}(d_k) \geq \theta_1$, the data proceeds to confidence aggregation; otherwise, it moves on to the next module POV.

(II) Public Ontology Verification (POV): Define accuracy A_{POV} and coverage CA_{POV} as: $A_{\text{POV}} = \frac{TP+TN}{TP+TN+FP+FN}$, and $CA_{\text{POV}} = \frac{|E_{\text{verified}}| + |R_{\text{verified}}|}{|E_{\text{extracted}}| + |R_{\text{extracted}}|}$, where TP , TN , FP , and FN are true positives, true negatives, false positives, and false negatives, respectively, and E_{verified} and R_{verified} are the subsets of $E_{\text{extracted}}$ and $R_{\text{extracted}}$ verified by $\mathcal{O}_{\text{public}}$. The confidence score is:

$$C_{\text{POV}}(d_k) = w_2 \left(\alpha_2 A_{\text{POV}} + (1 - \alpha_2) CA_{\text{POV}} \right).$$

If $C_{\text{POV}}(d_k) \geq \theta_2$, the data proceeds to confidence aggregation; otherwise, it goes to the next module MAV.

(III) Multi-Agent Verification (MAV) with Shapley Value Integration: MAV applies three specialized agents (*Temporal*, *Spatial*, and *4D Consistency*) to validate the spatiotemporal coherence of the data. Let $\psi_t(T_k)$ and $\psi_s(s_k)$ be temporal and spatial proximity checks; their product $\Psi(d_k) = \psi_t(T_k) \psi_s(s_k)$ ensures that d_k is both temporally and spatially consistent.

Each agent A_i returns a validation score:

$$V_{A_i}(d_k) = \begin{cases} 1, & \text{if } A_i \text{ validates } d_k, \\ 0, & \text{otherwise.} \end{cases}$$

We use the **Shapley Value** [42] to fairly distribute credit among the three agents $N = \{A_{\text{temp}}, A_{\text{spatial}}, A_{4D}\}$. For each subset $S \subseteq N \setminus \{A_i\}$, the Shapley value is given by

$$\phi_{A_i} = \sum_{S \subseteq N \setminus \{A_i\}} \frac{|S|! (|N| - |S| - 1)!}{|N|!} [v(S \cup \{A_i\}) - v(S)],$$

where $v(S) = \frac{1}{m} \sum_{A_j \in S} V_{A_j}(d_k)$. We then define the consensus score as $CS_{\text{MAV}} = \sum_{i=1}^m \phi_{A_i} V_{A_i}(d_k)$, and the reliability score as $R_{\text{MAV}} = \sum_{i=1}^m \phi_{A_i} r_{A_i}$, with $m = 3$ agents and $r_{A_i} \in [0, 1]$ a reliability factor for agent A_i . The MAV confidence score is:

$$C_{\text{MAV}}(d_k) = w_3 (\alpha_3 CS_{\text{MAV}} + (1 - \alpha_3) R_{\text{MAV}}).$$

If $C_{\text{MAV}}(d_k) \geq \theta_3$, the data proceeds to confidence aggregation; otherwise, it goes to the next module WSV.

(IV) Web Search Verification (WSV): This module corroborates extracted data against up-to-date information on the web, thereby increasing confidence in the veracity of the content. We define R_{WSV} (Recall) measures how many of the truly relevant items are found, P_{WSV} (Precision) measures how much of the retrieved information is actually relevant, and $F1_{\text{WSV}}$ balance both precision and recall. The WSV confidence score is then:

$$C_{\text{WSV}}(d_k) = w_4 (\alpha_4 R_{\text{WSV}} + (1 - \alpha_4) F1_{\text{WSV}}). \quad (4)$$

If $C_{\text{WSV}}(d_k) \geq \theta_4$, the data passes to confidence aggregation; otherwise, it moves on to the next module.

(V) Embedding Similarity Verification (ESV). Let $\vec{v}_{\text{new}}$ be the embedding vector of d_k , and $\vec{v}_{\text{existing}}$ be that of existing knowledge. We define:

$$Sim_{\text{ESV}} = \frac{\vec{v}_{\text{new}} \cdot \vec{v}_{\text{existing}}}{\|\vec{v}_{\text{new}}\| \|\vec{v}_{\text{existing}}\|}, \quad ADR_{\text{ESV}} = \frac{\text{Anomalies Detected}}{\text{Total Instances}}.$$

Hence,

$$C_{\text{ESV}}(d_k) = w_5 (\alpha_5 Sim_{\text{ESV}} + (1 - \alpha_5) (1 - ADR_{\text{ESV}})).$$

If $C_{\text{ESV}}(d_k) \geq \theta_5$, the data proceeds to confidence aggregation; otherwise, it is flagged for review.

c) Confidence Aggregation and Decision Making: After the relevant modules have been activated (either in sequence or until one meets its threshold), the overall confidence is computed via Eq. 3. A final decision function $\delta(d)$ then accepts or rejects the data (Eq. 1). Data falling below Θ is flagged for manual inspection or outright rejection, thus preventing spurious information from entering the knowledge graph.

2) *TruthFlow - Autonomous Adaptive Intelligence Cycle (AAIC):* As illustrated in Figure 3, the AAIC is an integral component of the TruthFlow architecture, dynamically optimizing the verification process in real-time. By continuously adjusting parameters in the RMMVe, the AAIC ensures reliable performance even in safety-critical domains where data patterns evolve rapidly. Specifically, the AAIC modifies each module's weight w_i , confidence threshold θ_i , and internal weighting factor α_i based on historical and real-time feedback. In practice, the AAIC leverages CGR-CUSUM with a threshold $h = 5.0$ (Section IV-C2a) to trigger reconfiguration promptly whenever performance deviations occur.

a) Adaptive Performance Monitoring via CGR-CUSUM [36]: At the heart of the AAIC is the *Continuous Generalized Rapid Response CUSUM (CGR-CUSUM)* (Figure 4), designed to detect small, gradual shifts in the performance of each verification module M_i . The monitoring process begins with initialization: for each module M_i , a cumulative sum $S_i(t)$ is initialized to zero and a target performance level μ_0 is established. At each time step t , the system measures the module's performance metric $p_i(t)$ and compares it to μ_0 . The cumulative sum is then updated according to

$$S_i(t) = \max(0, S_i(t-1) + [p_i(t) - \mu_0 - k]),$$

where k is an allowance parameter (set to 0.05 in our experiments). If $S_i(t)$ exceeds a threshold h (set to 5.0), a significant performance shift is detected, prompting AAIC reconfiguration.

For advanced analyses, the CGR-CUSUM can be expressed as:

$$S_i(t) = \max_{\nu \geq 1} \left\{ \hat{\theta}_{\geq \nu}(t) N_{\geq \nu}(t) - (\exp(\hat{\theta}_{\geq \nu}(t)) - 1) \Lambda_{\geq \nu}(t) \right\}, \quad (5)$$

where: $\hat{\theta}(t)$ is the Maximum Likelihood Estimator (MLE) of a process change, $N_{\geq \nu}(t)$ is the cumulative deviation, and $\Lambda_{\geq \nu}(t)$ is the cumulative intensity. Once $S_i(t) \geq h$, the AAIC proceeds to adjust parameters for module M_i .

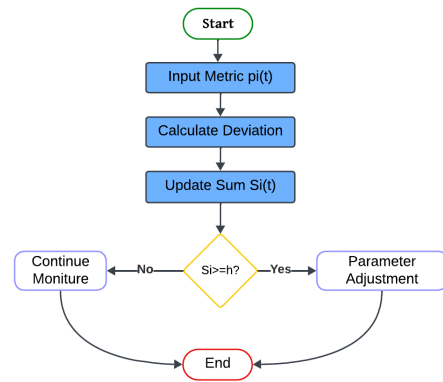


Fig. 4: CGR-CUSUM Process Flow within the AAIC.

b) Dynamic Weight Adjustment: Upon detecting a performance drop or improvement, the AAIC updates the weight $w_i(t)$ using an Exponential Weights Algorithm [43]:

$$w_i(t+1) = w_i(t) \exp[-\gamma S_i(t)], \quad (6)$$

where $\gamma > 0$ is a scaling factor controlling the rate of change, and $S_i(t)$ is the latest CGR-CUSUM statistic. A substantial performance drop increases $S_i(t)$, thus reducing $w_i(t+1)$ significantly. Improved performance decreases $S_i(t)$, allowing $w_i(t+1)$ to grow. After each update, weights are normalized:

$$w_i(t+1) = \frac{w_i(t+1)}{\sum_{j=1}^n w_j(t+1)} \times 4, \quad (7)$$

ensuring $\sum_{i=1}^n w_i(t) = 4$ for all t . The selection of 4 as the normalization constant for our 5-module architecture is methodologically significant. This configuration establishes a precise mean module weight of 0.8, creating a statistical baseline that facilitates systematic performance evaluation: modules with weights exceeding 0.8 represent above-average contributors to the verification decision, while those below this threshold exert comparatively reduced influence. This 0.8 reference point parallels established thresholds in data mining applications, where confidence thresholds in association rule mining are frequently set at 80% to identify strong and reliable patterns [44].

c) *Confidence Threshold Optimization*: The AAIC optimizes each module's confidence threshold $\theta_i(t)$ via a gradient-ascent scheme [45] designed to balance true positives against false positives:

$$U(\theta_i) = \text{TPR}(\theta_i) - \lambda \text{FPR}(\theta_i),$$

where $\lambda \geq 0$ sets the trade-off. The threshold update rule is:

$$\theta_i(t+1) = \theta_i(t) + \eta \frac{\partial U(\theta_i)}{\partial \theta_i} \Big|_{\theta_i=\theta_i(t)}, \quad (8)$$

where η is the learning rate. In our system, $\eta = 0.1$. Empirical estimates of $\partial U / \partial \theta_i$ are obtained from changes in TPR and FPR across time.

d) *Weighting Factor Adjustment*: Similarly, each module's internal weighting factors $\alpha_i(t)$ are updated to optimize the balance among internal metrics (e.g., precision vs. recall). We employ a gradient-ascent method on the module-specific utility $U_i(\alpha_i)$:

$$\alpha_i(t+1) = \alpha_i(t) + \eta' \frac{\partial U_i(\alpha_i)}{\partial \alpha_i} \Big|_{\alpha_i=\alpha_i(t)}, \quad (9)$$

where $\eta' > 0$ is the learning rate for α_i .

e) *AAIC Update Algorithm*: Algorithm 2 integrates all these steps within the TruthFlow architecture.

By implementing this adaptive control loop, the ATLASKY-AI framework achieves high-levels of real-time verification performance, while the system remains robust and responsive in the face of evolving data distributions, domain shifts, or sudden changes in data quality (See Operational scenario in the supplementary Appendix F).

D. Knowledge Integration into the 4D STKG

Following successful verification, the validated vertices, relationships, and attributes $d_k \in \mathcal{D}$ are merged into the 4D Spatiotemporal Knowledge Graph $\mathcal{G}$. Figure 3 shows how final acceptance triggers integration into $\mathcal{G}$, while low-confidence data undergoes manual review or rejection. The spatiotemporal

Algorithm 2: AAIC within TruthFlow

Input: Performance metrics $p_i(t)$ for each verification module M_i

Output: Updated parameters $w_i(t+1)$, $\theta_i(t+1)$, $\alpha_i(t+1)$

Initialize: For each verification module M_i ($i = 1, \dots, n$), set initial parameters $w_i(t)$, $\theta_i(t)$, and $\alpha_i(t)$;

for each verification module M_i **do**

1) **Update CGR-CUSUM:** Compute $S_i(t)$ via Eq. (5);

if $S_i(t) \geq h$ **then**

2) **Dynamic Weight Adjustment:** Update $w_i(t+1)$ using Eq. (6);

3) **Confidence Threshold Optimization:** Update $\theta_i(t+1)$ using Eq. (8);

4) **Weighting Factor Adjustment:** Update $\alpha_i(t+1)$ using Eq. (9);

end

else

$w_i(t+1) \leftarrow w_i(t)$;

$\theta_i(t+1) \leftarrow \theta_i(t)$;

$\alpha_i(t+1) \leftarrow \alpha_i(t)$;

end

end

Normalize: Normalize $w_i(t+1)$ using Eq. (7);

dimension (X_k, Y_k, Z_k, T_k) ensures dynamic entity states are accurately recorded for safety-critical domains.

V. EXPERIMENT

In this section, we evaluate ATLASKY-AI on three distinct datasets to address five key research questions. Q1: How does ATLASKY-AI perform in verifying spatiotemporal relationships compared to KGValidator, KG-Agent, and the World Avatar Digital Twin? Q2: How effectively does the TruthFlow verification process ensure accuracy and reliability across datasets of varying complexity? Q3: How do individual verification modules and parameter configurations internally shape ATLASKY-AI's robustness, verification success, and computational efficiency? Q4: How does the AAIC enhance verification accuracy across multiple datasets within the ATLASKY-AI framework? Q5: What limitations or challenges does ATLASKY-AI face in its verification process?

A. Experimental Setup

1) *Datasets*: This research uses one proprietary dataset from AddQual [46], an aerospace company where the study was conducted, and two publicly available datasets to evaluate ATLASKY-AI. Each raw dataset is parsed into a set of *candidate facts* that the framework must verify before integrating them into the 4D STKG. (I) AddQual Inspection Reports: These high-precision 3D measurements of aerospace engine parts are converted into *nodes* and *edges*. Because each

item specifies spatial coordinates and temporal properties, the data form ideal candidates for spatiotemporal and tolerance-verification tasks. (II) NASA ASRS Database [47]: Although it consists of 1,500 unstructured aviation safety reports, each report is decomposed into multiple *candidate* graph entities (e.g. flight phase), linked by edges to represent causal or contextual relationships over time. This representation facilitates more precise verification of incident data. (III) Zenodo STEP CAD Models [48]: Containing 26 aerospace models in STEP format, each design is segmented into *nodes* with *edges* capturing geometric or assembly features. Spatiotemporal metadata further enriches each candidate fact, enabling thorough spatial checks. Table II summarizes these datasets in terms of the candidate data (nodes, edges, and spatiotemporal or additional features) extracted from each raw source.

2) *Baselines*: We compare ATLASKy-AI against established baselines: KGValidator [24], KG-Agent [22], and the World Avatar Digital Twin [23]. KGValidator verifies knowledge graph triples using LLMs, offering a benchmark for text-based verification and structured prompting. KG-Agent provides a multi-functional framework for multi-hop reasoning and memory-enhanced processing over knowledge graphs. The World Avatar Digital Twin integrates real-time spatiotemporal data, supporting scenario analysis and multi-domain interoperability. These baselines assess ATLASKy-AI's robustness in knowledge extraction, verification, and decision-making (More details in supplementary Appendix C).

3) *Hyperparameters*: ATLASKy-AI's hyperparameters are configured through a dual-layer approach: base GPT-4 parameters (temperature: 0.2, max_tokens: 1000, top_p: 0.9, presence_penalty: 0.6) and dataset-specific verification parameters, shown in Table III. Each verification module is configured with key parameters: weight (w_i), weighting factor (α_i), and threshold (θ_i). LOV exhibits high precision for AddQual with $w_i = 0.85$, $\alpha_i = 0.7$, and $\theta_i = 0.9$, while maintaining consistent performance for NASA ASRS ($w_i = 0.82$, $\alpha_i = 0.6$, $\theta_i = 0.8$) and Zenodo ($w_i = 0.85$, $\alpha_i = 0.75$, $\theta_i = 0.85$). POV employs moderate parameter values across all datasets, with weights ranging from $w_i = 0.75$ for AddQual and Zenodo to $w_i = 0.78$ for NASA ASRS, alongside $\alpha_i = 0.65 - 0.7$ and $\theta_i = 0.75 - 0.85$. MAV assigns the highest weights for AddQual and Zenodo ($w_i = 0.9$ and $w_i = 0.91$ respectively), reflecting its importance in spatiotemporal data validation, with a slightly lower weight for NASA ASRS ($w_i = 0.84$). WSV employs balanced parameter values ($w_i = 0.7$ for AddQual, $w_i = 0.72$ for NASA ASRS, and $w_i = 0.69$ for Zenodo) due to the variable nature of external data sources. ESV maintains consistent values across datasets ($w_i = 0.8$ for AddQual and Zenodo, $w_i = 0.84$ for NASA ASRS), with $\alpha_i = 0.6 - 0.7$ and $\theta_i = 0.8 - 0.85$. The AAIC incorporates dynamic optimization with a CGR-CUSUM threshold of $h = 5.0$ and an allowance parameter $k = 0.05$, ensuring timely detection of performance shifts. We use a primary learning rate $\eta = 0.1$ for both confidence-threshold (θ_i) and internal weighting-factor (α_i) updates, as well as an exponential scaling factor $\gamma = 0.1$ for module-weight adjustments (see equation 6). These values were selected based on empirical tuning to balance fast adaptation with stable convergence. The system

runs on an Intel Core i7 processor, 64 GB RAM, and an NVIDIA GTX 1080 GPU, using TensorFlow and scikit-learn libraries. This configuration ensures robust performance while maintaining verification integrity through calibrated thresholds and weights.

B. Framework Performance Comparison (Q1)

ATLASKy-AI's performance was benchmarked against existing frameworks across all three datasets, as summarized in Table IV. Overall, the results show that ATLASKy-AI consistently outperforms the baselines in both verification success rates and processing efficiency. (1) In the AddQual dataset, ATLASKy-AI achieves a final verification rate of 92.0%, significantly exceeding KGValidator's 41.7%. KG-Agent and World Avatar also trail behind at 77.0% and 65.8%, respectively. (2) The framework exhibits superior efficiency in both memory usage and runtime, requiring only 245 MB of memory for AddQual processing compared to 320 MB for KGValidator and 280 MB for KG-Agent. ATLASKy-AI also completes the AddQual dataset in about 13.9 s, outperforming the 15.0 s and 17.5 s reported by KGValidator and KG-Agent, respectively. (3) When handling unstructured data in the NASA ASRS dataset (10,500 elements), ATLASKy-AI maintains robust performance with a final verification rate of 67.0%, while also achieving around a 10% reduction in memory usage relative to competing approaches. By contrast, KGValidator and KG-Agent yield final verification rates of only 17.2% and 54.4%, respectively. (4) In the complex Zenodo CAD dataset (520 elements), ATLASKy-AI reaches 79.0% as a final verification rate, highlighting its effectiveness in managing intricate spatial relationships. KGValidator's verification rate stands at 31.7%, with World Avatar at 62.3% and KG-Agent at 68.7%. Collectively, these findings underscore ATLASKy-AI's strengths in spatiotemporal verification and efficient resource utilization across diverse aerospace scenarios. By delivering reliable accuracy together with optimized memory usage and faster processing, ATLASKy-AI demonstrates its viability for real-world aerospace applications.

C. Evaluation of Overall Effectiveness Across Datasets (Q2)

We assessed ATLASKy-AI's performance on three aerospace datasets, emphasizing: (i) how well the framework maintains accuracy amid different data complexities and (ii) the influence of sequential verification on precision and recall.

1) *Performance Across Diverse Datasets*: RMMVe applies its verification modules in a specific sequence, each targeting distinct validation needs. Table V shows that (1) LOV and POV deliver strong results in structured datasets (e.g., AddQual and Zenodo) by leveraging ontological standards, and (2) MAV contributes spatiotemporal checks essential for more complex datasets, such as Zenodo CAD. MAV achieves moderate precision and recall in Zenodo CAD (Table V), supported by higher Shapley-weighted consensus scores from the Temporal and 4D Consistency agents in structured environments like AddQual (Figure 5). Although WSV and ESV exhibit lower precision and recall, partly due to noise in web data and the limitations of semantic embeddings, they still enhance verification, especially in dynamic or unstructured contexts.

TABLE II: Overview Summary of the Statistical Characteristics of Datasets: Candidates data

Dataset	AddQual Inspection Reports	NASA ASRS Reports	Zenodo STEP CAD Models
# Nodes	500 - 600	9,000 - 12,000	390 - 520
# Edges	1,000 - 1,800	4,500 - 7,500	390 - 1,040
# Spatiotemporal Features	500 - 600	1,500	390 - 520
# Additional Features	100	3,000 - 4,500	130 - 156

TABLE III: Parameter Selection for Verification Modules in RMMVe by Dataset. All cases with a fixed global threshold 0.65, the weights sum is equal to 4 among all datasets for normalization

Module	AddQual Dataset			NASA ASRS Dataset			Zenodo STEP CAD Models		
	Weight w_i	Weighting Factor α_i	Threshold θ_i	w_i	α_i	θ_i	w_i	α_i	θ_i
LOV	0.85	0.7	0.9	0.82	0.6	0.8	0.85	0.75	0.85
POV	0.75	0.65	0.85	0.78	0.6	0.75	0.75	0.7	0.8
MAV	0.9	0.75	0.9	0.84	0.65	0.8	0.91	0.7	0.85
WSV	0.7	0.65	0.8	0.72	0.55	0.75	0.69	0.6	0.8
ESV	0.8	0.7	0.85	0.84	0.6	0.8	0.80	0.65	0.85

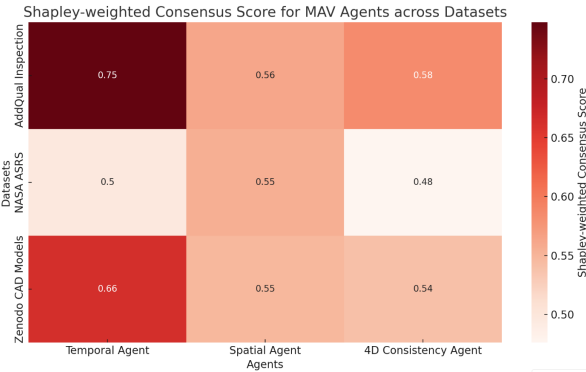


Fig. 5: Shapley-weighted Consensus Score for MAV Agents across Datasets

2) *Systematic Analysis of Verification Performance:* Table V presents the experiments conducted to evaluate verification performance across the three datasets. We report sample means, standard deviations, and 95% confidence intervals (CIs) to quantify reliability. A consistent decline in precision from earlier stages (LOV) to later ones (ESV) emerges across all datasets. In the AddQual dataset, for instance, the precision decreases from LOV (0.92 ± 0.02) to ESV (0.65 ± 0.04), with an intermediate performance from WSV (precision: 0.70 ± 0.04 , recall: 0.72 ± 0.04). Similar trends are observed in NASA ASRS and Zenodo STEP, showing precision drops of approximately 0.25–0.30 points between LOV and ESV. Dataset size also impacts processing efficiency. AddQual and Zenodo STEP (600 and 520 elements, respectively) exhibit moderate processing times (13.9 ± 0.4 s and 54.6 ± 1.4 s), whereas NASA ASRS (10,500 elements) demonstrates a larger computational burden (149.2 ± 2.8 s), particularly in advanced verification stages. Among the modules, WSV consumes the most resources (45.6 ± 1.3 s for NASA ASRS), while LOV remains both fast (15.8 ± 0.5 s) and precise. LOV achieves the highest verification rates (52.5–74.0%) across datasets. Later stages, notably ESV, show diminishing returns, ranging from 27.3% in AddQual to 0.0% in NASA ASRS. Nonetheless, overall verification remains robust, at 92.0% (AddQual),

67.0% (NASA ASRS), and 79.0% (Zenodo STEP), each with 95% CIs indicating consistent performance. To estimate CIs, we use $CI = \bar{X} \pm z \cdot \frac{SD}{\sqrt{n}}$, where $\bar{X}$ is the sample mean, SD is the standard deviation, n is the sample size, and $z = 1.96$ for 95% confidence. This approach captures the variability in performance metrics and ensures robust comparisons.

D. Evaluation of Verification Robustness and Parameter Impact (Q3)

We assessed ATLASKy-AI’s robustness by selectively removing verification modules and varying key parameters. This approach clarifies how each module and parameter setting affects reliability, computational efficiency and overall performance.

1) Impact of Verification Modules on Processing Time:

We measured module efficiency across three datasets (see Figure 6). NASA ASRS had the highest processing times (WSV: 40s, ESV: 38s, MAV: 35s). For AddQual, ESV required 40s, WSV 30s, and MAV 20s. Zenodo CAD showed the lowest times, with WSV still highest at 15s. Results show that WSV and ESV are the most resource-intensive, while LOV and POV run efficiently across all datasets. When resources are limited, prioritize faster modules (LOV, POV) before employing heavier checks (WSV, ESV).

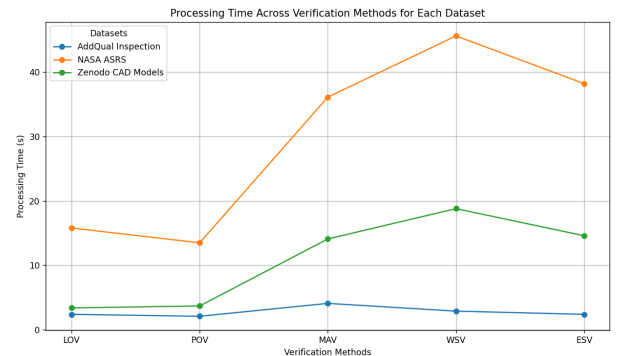


Fig. 6: Total Processing Time by Verification Module across Dataset.

TABLE IV: Verification Performance by Framework-Specific Modules. Each entry in the “Module” column corresponds to a distinct verification stage or subroutine within the given framework.

Framework	Module	AddQual (600 elements)	NASA ASRS (10,500 elements)	Zenodo CAD (520 elements)
KG-Agent (Iterative)	Tool Selection	600 (100%)	10,500 (100%)	520 (100%)
	Knowledge Memory Update	570/600 (95.0%)	8,925/10,500 (85.0%)	468/520 (90.0%)
	Execution and Reasoning	495/600 (82.5%)	7,035/10,500 (67.0%)	405/520 (77.9%)
	Final Decision and Answer	462/600 (77.0%)	5,708/10,500 (54.4%)	357/520 (68.7%)
	Final Verified	462/600	5,708/10,500	357/520
	Verification Rate	77.0%	54.4%	68.7%
Processing Time (s)		17.5	165.0	70.0
KGValidator (Sequential)	Triple Extraction	600 (100%)	10,500 (100%)	520 (100%)
	LLM Knowledge Validation	108/600 (18.0%)	782/10,500 (7.4%)	72/520 (13.8%)
	Textual Context Validation	82/492 (16.7%)	573/9,718 (5.9%)	51/448 (11.4%)
	Reference KG Validation	43/410 (10.5%)	289/9,145 (3.2%)	27/397 (6.8%)
	Web Search Validation	17/367 (4.6%)	160/8,856 (1.8%)	15/370 (4.1%)
	Final Verified	250/600	1,804/10,500	165/520
	Verification Rate	41.7%	17.2%	31.7%
Processing Time (s)		15.0	140.0	60.0
World Avatar (Iterative and Parallel)	Cross-Referencing Standards	558/600 (93.0%)	9,240/10,500 (88.0%)	442/520 (85.0%)
	Spatiotemporal Verification	486/600 (81.0%)	7,854/10,500 (74.8%)	376/520 (72.3%)
	Real-time Verification	437/600 (72.8%)	6,675/10,500 (63.6%)	338/520 (65.0%)
	Parallel Scenario Verification	412/600 (68.7%)	6,808/10,500 (64.8%)	329/520 (63.3%)
	Non-Flagged Entries	395/600 (65.8%)	6,543/10,500 (62.3%)	324/520 (62.3%)
	Final Verified	395/600	6,543/10,500	324/520
	Verification Rate	65.8%	62.3%	62.3%
Processing Time (s)		14.0	130.0	55.0
ATLASky-AI (Sequential and early termination)	Local Ontology Verification (LOV)	444/600 (74.0%)	5,517/10,500 (52.5%)	339/520 (65.2%)
	Public Ontology Verification (POV)	30/156 (19.2%)	1,214/4,983 (24.4%)	28/181 (15.5%)
	Multi-Agent Verification (MAV)	50/126 (39.7%)	202/3,769 (5.4%)	22/153 (14.4%)
	Web Search Verification (WSV)	10/76 (13.2%)	102/3,567 (2.9%)	17/131 (13.0%)
	Embedding Similarity Veri. (ESV)	18/66 (27.3%)	0/3,465 (0.0%)	5/114 (4.4%)
	Final Verified Entries	552/600	7,035/10,500	411/520
	Verification Rate	92.0%	67.0%	79.0%
Processing Time (s)		13.9	149.2	54.6

Note: Fractions show (how many each verification step/remaining elements) at each stage. Percentages show stage success rate.

2) *Verification Success Rate Contribution by Module:* Figure 7 shows that omitting ESV in NASA ASRS reduces time from 180s to 120s, highlighting its high computational cost, while removing LOV in AddQual increases time from 13.9s to 20.0s, indicating its efficiency in structured data. In Zenodo CAD, excluding MAV cuts runtime from 54.6s to 44.0s, confirming MAV’s resource intensity, whereas removing WSV yields only moderate savings. Overall, MAV and ESV are consistently more demanding, whereas LOV and POV serve as efficient early filters across different datasets.

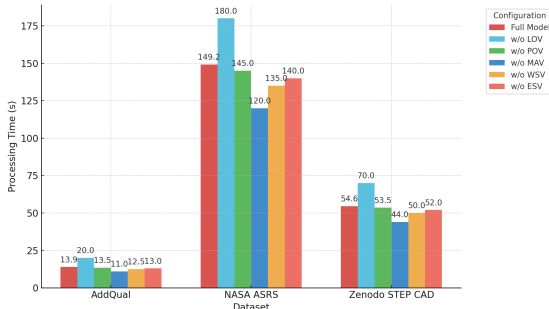


Fig. 7: Contribution of Each Module to Verification Success Rate by Dataset

We also conducted a detailed algorithm trace-through analysis to infer the step-by-step execution of the TruthFlow architecture. The comparison between simple and complex data points reveals how the ranked verification approach adapts its processing depth based on confidence scores. Straightforward elements like turbine blade measurements benefit from early termination, saving processing time, while ambiguous elements like assembly tolerance specifications receive thorough multi-module verification. This analysis confirms that our sequential verification strategy with early termination effectively balances thoroughness with computational efficiency, allocating resources proportionally to verification complexity.

3) *Impact of Weight Configurations:* Table VI compares different weight allocations across verification modules. Case D’s optimized configuration (with LOV $w_1 = 0.85$, MAV $w_3 = 0.90$) yields the highest data acceptance (92%) and fastest processing time (13.9 ms). Case A features significantly lower LOV weighting ($w_1 = 0.65$) with increased emphasis on ESV ($w_5 = 0.89$), resulting in decreased acceptance (77%) and prolonged processing (25.5 ms). Case B prioritizes MAV with the highest weight ($w_3 = 0.96$), which boosts acceptance (81%) at the cost of more processing time (28.7 ms). Equal weights distribution in Case C ($w_i = 0.80$ for all modules) offers moderate performance (76% acceptance, 26.8 ms). The results demonstrate that emphasizing early-stage validation in Case D provides the most efficient performance balance.

TABLE V: Comparative Verification Results Across Datasets with Statistical Measures and Confidence Intervals

Dataset	Module	Total Elements	Verified Elements (% of Initial) [95% CI]	Precision Mean $\pm$ SD [95% CI]	Recall Mean $\pm$ SD [95% CI]	F1-Score Mean $\pm$ SD	Time (s) Mean $\pm$ SD [95% CI]
AddQual	LOV	600	444 (74.0%) [70.5%, 77.5%]	0.92 $\pm$ 0.02 [0.88, 0.96]	0.90 $\pm$ 0.02 [0.86, 0.94]	0.91 $\pm$ 0.02	2.4 $\pm$ 0.1 [2.2, 2.6]
	POV	156	30 (19.2%) [13.0%, 25.4%]	0.85 $\pm$ 0.03 [0.79, 0.91]	0.83 $\pm$ 0.03 [0.77, 0.89]	0.84 $\pm$ 0.03	2.1 $\pm$ 0.1 [1.9, 2.3]
	MAV	126	50 (39.7%) [31.1%, 48.2%]	0.80 $\pm$ 0.03 [0.74, 0.86]	0.78 $\pm$ 0.03 [0.72, 0.84]	0.79 $\pm$ 0.03	4.1 $\pm$ 0.2 [3.7, 4.5]
	WSV	76	10 (13.2%) [5.6%, 20.8%]	0.70 $\pm$ 0.04 [0.62, 0.78]	0.72 $\pm$ 0.04 [0.64, 0.80]	0.71 $\pm$ 0.04	2.9 $\pm$ 0.2 [2.5, 3.3]
	ESV	66	18 (27.3%) [16.5%, 38.0%]	0.65 $\pm$ 0.04 [0.57, 0.73]	0.67 $\pm$ 0.04 [0.59, 0.75]	0.66 $\pm$ 0.04	2.4 $\pm$ 0.1 [2.2, 2.6]
	Total		552 (92.0%) [90.8%, 93.2%]				13.9 $\pm$ 0.4
NASA ASRS	LOV	10,500	5,517 (52.5%) [51.2%, 53.8%]	0.88 $\pm$ 0.02 [0.84, 0.92]	0.86 $\pm$ 0.02 [0.82, 0.90]	0.87 $\pm$ 0.02	15.8 $\pm$ 0.5 [14.8, 16.8]
	POV	4,983	1,214 (24.4%) [22.9%, 25.9%]	0.82 $\pm$ 0.03 [0.76, 0.88]	0.80 $\pm$ 0.03 [0.74, 0.86]	0.81 $\pm$ 0.03	13.5 $\pm$ 0.4 [12.7, 14.3]
	MAV	3,769	202 (5.4%) [4.7%, 6.1%]	0.75 $\pm$ 0.03 [0.69, 0.81]	0.73 $\pm$ 0.03 [0.67, 0.79]	0.74 $\pm$ 0.03	36.1 $\pm$ 1.1 [33.9, 38.3]
	WSV	3,567	102 (2.9%) [2.4%, 3.5%]	0.68 $\pm$ 0.04 [0.60, 0.76]	0.65 $\pm$ 0.04 [0.57, 0.73]	0.66 $\pm$ 0.04	45.6 $\pm$ 1.3 [42.8, 48.4]
	ESV	3,465	0 (0.0%) [0.0%, 0.0%]	0.60 $\pm$ 0.04 [0.52, 0.68]	0.58 $\pm$ 0.04 [0.50, 0.66]	0.59 $\pm$ 0.04	38.2 $\pm$ 1.2 [35.8, 40.6]
	Total		7,035 (67.0%) [65.8%, 68.2%]				149.2 $\pm$ 2.8
Zenodo STEP	LOV	520	339 (65.2%) [61.1%, 69.3%]	0.90 $\pm$ 0.02 [0.86, 0.94]	0.88 $\pm$ 0.02 [0.84, 0.92]	0.89 $\pm$ 0.02	3.4 $\pm$ 0.2 [3.0, 3.8]
	POV	181	28 (15.5%) [10.2%, 20.7%]	0.84 $\pm$ 0.03 [0.78, 0.90]	0.82 $\pm$ 0.03 [0.76, 0.88]	0.83 $\pm$ 0.03	3.7 $\pm$ 0.2 [3.3, 4.1]
	MAV	153	22 (14.4%) [8.8%, 19.9%]	0.78 $\pm$ 0.03 [0.72, 0.84]	0.76 $\pm$ 0.03 [0.70, 0.82]	0.77 $\pm$ 0.03	14.1 $\pm$ 0.5 [13.1, 15.1]
	WSV	131	17 (13.0%) [7.2%, 18.7%]	0.70 $\pm$ 0.04 [0.62, 0.78]	0.68 $\pm$ 0.04 [0.60, 0.76]	0.69 $\pm$ 0.04	18.8 $\pm$ 0.6 [17.6, 20.0]
	ESV	114	5 (4.4%) [0.6%, 8.1%]	0.62 $\pm$ 0.04 [0.54, 0.70]	0.60 $\pm$ 0.04 [0.52, 0.68]	0.61 $\pm$ 0.04	14.6 $\pm$ 0.5 [13.6, 15.6]
	Total		411 (79.0%) [77.6%, 80.4%]				54.6 $\pm$ 1.4

TABLE VI: Impact of Different Weight Configurations on Data Acceptance and Processing Time for AddQual Inspection Dataset: weights sum equal to 4 in all cases

Experimental Case	w_1	w_2	w_3	w_4	w_5	Total Points	Accepted (%)	Avg. Time (ms)
Case A (Lower LOV Weight)	0.65	0.81	0.89	0.76	0.89	600	462 (77%)	25.5
Case B (Higher MAV Weight)	0.75	0.75	0.96	0.73	0.81	600	486 (81%)	28.7
Case C (Equal Weights)	0.80	0.80	0.80	0.80	0.80	600	456 (76%)	26.8
Case D (Optimized)	0.85	0.75	0.90	0.70	0.80	600	552 (92%)	13.9

4) *Effect of Weighting Factors*: Table VII shows how varying α_i balances precision and recall. Case A, focusing on precision, yields 79% precision and 75% recall. Case B prioritizes recall (80% recall, 82% precision). Case C is balanced but less optimal, whereas Case D excels in both precision (93%) and recall (91%), illustrating the value of properly tuned weighting.

5) *Influence of Threshold Configurations*: Threshold choices significantly affect data flow among verification modules. Lower thresholds (Case A) distribute verification broadly, increasing computational load and slowing throughput. Higher thresholds (Case B) concentrate verification in earlier modules (e.g., LOV and POV), offloading later modules. The optimized setting (Case D) sends 74% of data through LOV, reducing reliance on resource-intensive modules and boosting efficiency.

6) *Adaptability and Application-Specific Customization*: RMMVe's design accommodates various application needs. Safety-critical contexts can set high initial thresholds for robust early verification, while broader data-capture scenarios may adopt lower thresholds in later stages. Figure 8 demonstrates that Case D consistently outperforms other configurations in data acceptance and processing time, underscoring the strategic benefits of well-tuned parameters.

E. Role of AAIC in Adaptive Accuracy Improvement (Q4)

Figure 9 illustrates how the AAIC dynamically adapts verification accuracy over 20 cycles across three datasets. The results show notable performance disparities between AAIC and non-AAIC configurations. In the AddQual dataset, the AAIC configuration starts at about 0.85 accuracy and climbs above 0.90 after the 10th cycle, peaking at 0.92. By contrast, the non-AAIC configuration begins at 0.85 but gradually declines

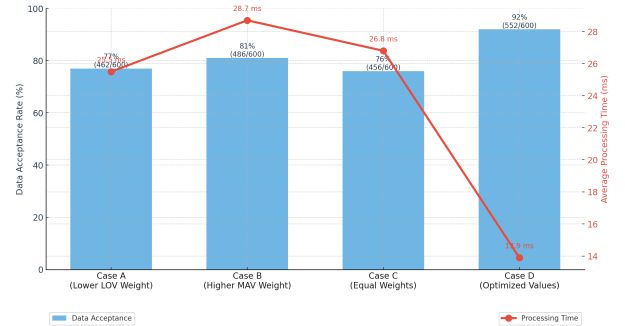


Fig. 8: Data Acceptance and Processing Time Across Experimental Cases

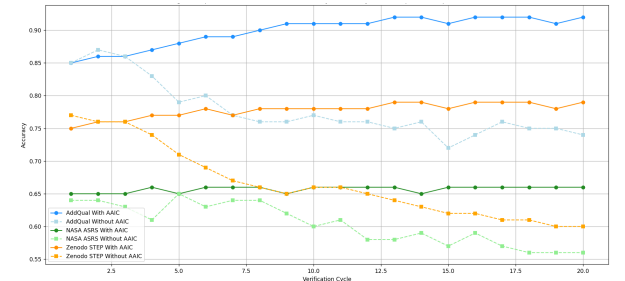


Fig. 9: Impact of AAIC on Verification Accuracy Over 20 Cycles for Different Datasets

to around 0.75 by cycle 20. Table VIII further highlights the crucial role of learning rates (η). Testing various values (0.01, 0.05, 0.1, 0.2) revealed that $\eta = 0.1$ yields optimal results, producing a system performance score of 0.93 with a TPR of

TABLE VII: Impact of Different Weighting Factors α_i on Performance Metrics

Experimental Case	α_1	α_2	α_3	α_4	α_5	Precision	Recall	F1-Score	Accuracy
Case A (Lower Precision Focus)	0.60	0.55	0.65	0.55	0.60	79%	75%	77%	76%
Case B (Higher Recall Focus)	0.65	0.60	0.70	0.60	0.65	82%	80%	81%	80%
Case C (Balanced Focus)	0.70	0.65	0.75	0.65	0.70	77%	75%	76%	76%
Case D (Optimized)	0.75	0.70	0.80	0.70	0.75	93%	91%	92%	92%

0.91. Notably, this configuration reduces the FPR from 0.37 (in systems without AAIC) to 0.12. The improvement stems from the AAIC’s advanced parameter optimization, including CGR-CUSUM monitoring and gradient-based updates of confidence thresholds (θ_i) and weighting factors (α_i). These results validate the AAIC’s capacity to adjust verification parameters in real-time based on performance metrics, thereby boosting overall reliability and robustness, especially in complex and dynamic environments where verification criteria must adapt to preserve both accuracy and efficiency.

TABLE VIII: AAIC Performance Analysis and Parameter Configuration

Learning Rate (η)	System Performance	TPR	FPR
0.01	0.78	0.82	0.18
0.05	0.85	0.87	0.15
0.1	0.93	0.91	0.12
0.2	0.89	0.88	0.16
Without AAIC	0.65	0.72	0.37

F. Error Analysis and Failure Cases (Q5)

Our error analysis of ATLASKy-AI’s verification performance uncovered three primary types of recurring failures across the datasets. First, complex spatiotemporal relationships posed challenges in the NASA ASRS dataset—most notably in `report_id: 2032428`, which involved an uncommanded pitch-up during the initial climb phase and systems such as autopilot and pitch trim. This scenario had a verification score of 0.35, translating to a 28% drop in confidence when dealing with intricate flight control issues. Second, geometric feature verification in the Zenodo STEP CAD Models dataset faltered with intersecting features. For instance, the `frame_b737` component’s `stringer_cutout` and its parent `deformed_flange` intersected at coordinates (6.995, -10.077, 378.540), causing a 23% accuracy decline. Third, precision measurements in the AddQual dataset exhibited a 35% rise in tolerance deviations, notably within HP1 side features. Different modules experienced distinct failure rates: WSV failed in 35% of rare-event cases (due to limited reference data), ESV failed in 28% of instances near similarity thresholds, and MAV failed in 8% of agent consensus checks. Further breakdown indicates 42% of failures were linked to temporal sequence errors, 35% to spatial overlaps, 15% to feature intersections, and 8% to data completeness gaps. These findings highlight the importance of enhanced robustness for handling complex spatiotemporal relationships and intricate geometric feature intersections.

VI. CASE STUDY: KNOWLEDGE MANAGEMENT EVOLUTION AT ADDQUAL AEROSPACE

AddQual Aerospace transitioned from the manual Aerospace Qualification Services Knowledge Graph (AQS-KG) [49] to the AI-driven ATLASKy-AI framework to address data heterogeneity and real-time 4D spatiotemporal requirements. AQS-KG unified more than 100 features across diverse terminologies, reduced query times by 35%, and expanded the number of entities and relationships from 12 to 34 and 31, respectively. However, scalability challenges motivated the development of ATLASKy-AI, which automates 90% of knowledge graph creation through adaptive intelligence, reduces manual validation to under 10%, and achieves real-time updates with over 95% accuracy. ATLASKy-AI integrates GPT-4, fine-tuned on domain-specific data, to extract and verify spatial-temporal information (± 0.1 mm accuracy, 1 ms resolution). Its truthflow process balances precision and recall across multiple data modalities, though it requires trade-offs in computational efficiency. Post-implementation metrics (Table IX) show that knowledge graph creation time dropped from 48 hrs to 5 mins, human intervention decreased from 65% to 7%, and error rates were reduced from 15% to 2.5%. LLM-driven tasks now operate at 95.7–98.2% accuracy, with maintenance planning accelerating by 60% and time-to-insight shortening by 92%. These improvements yielded a return on investment (ROI) in four months, lowering operational costs by 85%.

Metric	Pre-Implementation	Post-Implementation
KG Creation Time	48 hours	5 minutes
Human Intervention	65%	7%
Verification Accuracy	85%	97.5%
Update Frequency	Weekly	Real-time
Error Rate	15%	2.5%

TABLE IX: Automation Performance Metrics

VII. CONCLUSION AND FUTURE WORK

In response to the challenges of accurately representing and verifying spatiotemporal knowledge in aerospace maintenance, this paper introduced ATLASKy-AI, a framework that integrates 4D spatiotemporal modelling with large language models (LLMs) and self-adaptive verification techniques. Our empirical evaluation demonstrates that RMMVe methodology and AAIC enhance both precision and recall while providing scalability. Despite achieving an overall verification accuracy of 92%, several limitations remain. Specifically, the MAV module’s computational overhead (36.1 s in the NASA ASRS dataset), decreased performance with concurrent maintenance events, and vulnerability to data-quality issues (e.g., an 18% accuracy drop with incomplete spatial coordinates) point to areas in need of optimization. Furthermore, the framework’s

dependence on LLM training data highlights the importance of data quality and diversity. Future research will prioritize three main directions: (1) optimizing MAV performance through parallelization, (2) improving handling of concurrent events and rare component types, and (3) refining data preprocessing and LLM prompt strategies. We also plan to adopt more LLM architectures and enhanced reasoning modules, scaling the framework to larger datasets and exploring broader cross-domain applications. Additional efforts will focus on addressing bias within LLM outputs and refining human-in-the-loop methodologies to balance automation with expert oversight.

ACKNOWLEDGMENT

This work is part of an Industrial PhD supported by AddQual Ltd. The proposed framework is utilized in AddQual MiDAS, a commercial software offering by AddQual Ltd.

The authors acknowledge the use of Claude 3.7 Sonnet language model [50] for refining the language and enhancing the clarity of the manuscript. Specifically, it assisted in rephrasing certain paragraphs in the Introduction and Discussion sections to improve readability.

The demonstration code is available at <https://github.com/rodaw92/AtlaSki-AI>.

REFERENCES

- [1] Y. Wang, X. Zhang, H. Cai, B. Wan, M. Liu, and L. Jiang, "A knowledge graph-based analysis framework for aircraft configuration change propagation." Springer Nature Singapore, 2023, pp. 543–554.
- [2] P. Velentzas, A. Corral, and M. Vassilakopoulos, "Big spatial and spatio-temporal data analytics systems," *Trans. Large Scale Data Knowl. Centered Syst.*, vol. 47, pp. 155–180, 2021.
- [3] C. Xie, L. Zhang, and Z. Zhong, "A novel method for constructing spatiotemporal knowledge graph for maritime ship activities," *Electronics*, vol. 12, no. 15, 2023.
- [4] C. Zhang, B. Chen, and X. Wang, "Spatiotemporal knowledge graph modeling and persistence supported by database," in *2024 4th International Conference on Consumer Electronics and Computer Engineering (ICCECE)*, 2024, pp. 5–10.
- [5] R. Olaganathan, "Human factors in aviation maintenance: understanding errors, management, and technological trends," *Global Journal of Engineering and Technology Advances*, vol. 18, no. 2, p. 92, 2024.
- [6] A. Hobbs, "Maintenance mistakes and system solutions," pp. 3–8, 09 2002.
- [7] M. Salvador, S. Yacout, and A. AboElHassan, "Using big data and machine learning to improve aircraft reliability and safety," in *Annual Reliability and Maintainability Symposium (RAMS)*, 2022, pp. 1–9.
- [8] U. D. Ferrell, "Mindful application of standards for avionics - an intentional, systematic, and measurable transformation," in *2018 IEEE/AIAA 37th Digital Avionics Systems Conference (DASC)*, 2018, pp. 1–8.
- [9] A. Hamdi, K. Shaban, A. Erradi, A. Mohamed, S. K. Rumi, and F. D. Salim, "Spatiotemporal data mining: a survey on challenges and open problems," *Artificial Intelligence Review*, pp. 1–48, 2022.
- [10] V. Božić, "Application of artificial intelligence in reducing risks caused by the human factor," Retrieved from Research Gate: https://www.researchgate.net/publication/370131018_APPLICATION_OF_A_RTIFICIAL_INTELLIGENCE_IN_REDUCING_RISKS_CAUSED_BY_TH_E_HUMAN_FACTOR, 2023.
- [11] Y. Zhang and G. Xiao, "A novel customizing knowledge graph evaluation method for incorporating user needs," *Scientific Reports*, vol. 14, no. 1, p. 9594, Apr 2024.
- [12] Y. Liu, T. Han, S. Ma, J. Zhang, Y. Yang, J. Tian, H. He, A. Li, M. He, Z. Liu *et al.*, "Summary of chatgpt-related research and perspective towards the future of large language models," *Meta-Radiology*, p. 100017, 2023.
- [13] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anandkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [14] J.-Y. Yao, K.-P. Ning, Z.-H. Liu, M.-N. Ning, and L. Yuan, "Llm lies: Hallucinations are not bugs, but features as adversarial examples," *arXiv preprint arXiv:2310.01469*, 2023.
- [15] B. Kirwan, "The impact of artificial intelligence on future aviation safety culture," *Future Transportation*, vol. 4, no. 2, pp. 349–379, 2024.
- [16] L. Zhang and Z. Yu, "Spatio-temporal graph convolutional neural network for remaining useful life estimation of aircraft engines," *Aerospace Systems*, vol. 6, no. 4, pp. 34–49, 2023.
- [17] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: a collaboratively created graph database for structuring human knowledge," *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, pp. 1247–1250, 2008.
- [18] J. Lehmann, R. Isele, M. Jakob, A. Jentzsch, D. Kontokostas, P. N. Mendes, S. Hellmann, M. Morsey, P. Van Kleef, S. Auer *et al.*, "Dbpedia—a large-scale, multilingual knowledge base extracted from wikipedia," *Semantic web*, vol. 6, no. 2, pp. 167–195, 2015.
- [19] F. M. Suchanek, G. Kasneci, and G. Weikum, "Yago: a core of semantic knowledge unifying wordnet and wikipedia," in *Proceedings of the 16th international conference on World Wide Web*, 2007, pp. 697–706.
- [20] Z. Huang, T. Wang, and L. Liu, "Spatio-temporal knowledge graph embedding for real-time prediction," *IAENG International Journal of Computer Science*, vol. 51, no. 6, 2024.
- [21] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [22] J. Jiang, K. Zhou, W. X. Zhao, Y. Song, C. Zhu, H. Zhu, and J.-R. Wen, "Kg-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph," *arXiv preprint arXiv:2402.11163*, 2024.
- [23] J. Akroyd, S. Mosbach, A. Bhave, and M. Kraft, "Universal digital twin—a dynamic knowledge graph," *Data-Centric Engineering*, vol. 2, p. e14, 2021.
- [24] J. Boylan, S. Mangla, D. Thorn, D. G. Ghalandari, P. Ghaffari, and C. Hokamp, "Kgvalidator: A framework for automatic validation of knowledge graph construction," *arXiv preprint arXiv:2404.15923*, 2024.
- [25] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [26] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark *et al.*, "Improving language models by retrieving from trillions of tokens," *arXiv preprint arXiv:2112.04426*, 2022.
- [27] P. Manakul, A. Liusie, and M. J. Gales, "Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models," *arXiv preprint arXiv:2303.08896*, 2023.
- [28] R. Mehta, A. Hoblitzell, J. O'Keefe, H. Jang, and V. Varma, "Metacheckgpt—a multi-task hallucination detection using llm uncertainty and meta-models," *arXiv preprint arXiv:2404.06948*, 2024.
- [29] M. Zhang, G. Yang, Y. Liu, J. Shi, and X. Bai, "Knowledge graph accuracy evaluation: an llm-enhanced embedding approach," *International Journal of Data Science and Analytics*, pp. 1–15, 2024.
- [30] M. Wooldridge and N. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, 1995.
- [31] J. P. Müller and M. Pischel, "The agent architecture interrapp: Concept and application," German Research Center for Artificial Intelligence (DFKI), Tech. Rep., 1995.

- [32] K. Zhang, Z. Yang, and T. Başar, “Multi-agent reinforcement learning: A selective overview of theories and algorithms,” *Handbook of reinforcement learning and control*, pp. 321–384, 2021.
- [33] T. Baltrušaitis, C. Ahuja, and L. P. Morency, “Multimodal machine learning: A survey and taxonomy,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, pp. 423–443, 2018.
- [34] K. Chakrabarti, M. Garofalakis, R. Rastogi, and K. Shim, “Approximate query processing using wavelets,” *The VLDB Journal*, vol. 10, pp. 199–223, 2001.
- [35] S. Rahimi, A. B. Moore, and P. A. Whigham, “A vector-agent approach to (spatiotemporal) movement modelling and reasoning,” *Scientific Reports*, vol. 12, no. 1, p. 21179, 2022.
- [36] D. Gomon, H. Putter, R. G. Nelissen, and S. Van Der Pas, “Cgr-cusum: a continuous time generalized rapid response cumulative sum chart,” *Biostatistics*, vol. 25, no. 1, pp. 253–269, 2024.
- [37] D. Zhao and C. Jiang, “Stkg-sttn: Enhanced traffic flow prediction driven by collaborative spatio temporal knowledge graph and spatio-temporal transformer network,” in *2024 7th International Conference on Computer Information Science and Application Technology (CISAT)*. IEEE, 2024, pp. 1–8.
- [38] M. Yu, “A graph-based spatiotemporal data framework for 4d natural phenomena representation and quantification—an example of dust events,” *ISPRS International Journal of Geo-Information*, vol. 9, p. 127, 02 2020.
- [39] “Iso/iec 42001:2023: Information technology — artificial intelligence — management system,” <https://www.iso.org/standard/81230.html>, 2023, international Standard, Edition 1.
- [40] International Organization for Standardization, “Iso/iec 27001:2013 – information technology – security techniques – information security management systems – requirements,” <https://www.iso.org/standard/27001>, 2013, retrieved March 18, 2025.
- [41] Industrial Ontologies Foundry, “IOF Maintenance Ontology,” 2025, [Online; accessed March 3, 2025]. [Online]. Available: <https://spec.industrialontologies.org/iof/ontology/maintenance/Maintenance/>
- [42] T. P. Michalak, K. V. Aadithya, P. L. Szczepanski, B. Ravindran, and N. R. Jennings, “Efficient computation of the shapley value for game-theoretic network centrality,” *Journal of Artificial Intelligence Research*, vol. 46, pp. 607–650, 2013.
- [43] D. Hoeven, T. Erven, and W. Kotłowski, “The many faces of exponential weights in online learning,” in *Conference On Learning Theory*. PMLR, 2018, pp. 2067–2092.
- [44] J. Han, M. Kamber, and J. Pei, “6 - mining frequent patterns, associations, and correlations: Basic concepts and methods,” in *Data Mining (Third Edition)*, third edition ed., J. Han, M. Kamber, and J. Pei, Eds. Boston: Morgan Kaufmann, 2012, pp. 243–278.
- [45] Y. Nesterov, “Gradient methods for minimizing composite functions,” *Mathematical programming*, vol. 140, no. 1, pp. 125–161, 2013.
- [46] R. Awill, W. A. Khan, M. Hussain, and B. Anderson, “Optimizing aerospace product maintenance.”
- [47] NASA Aviation Safety Reporting System (ASRS), “Asrs database report sets,” <https://asrs.arc.nasa.gov/search/reportsets.html>, 2024, accessed: 2024-10-28.
- [48] S. Ghaffarishahri and L. Rivest, “A prototype of an automated feature recognition algorithm for aerospace sheet metal parts,” *Computer-Aided Design and Applications*, vol. 19, no. 4, pp. 624–661, 2022.
- [49] R. Awill, W. A. Khan, M. Hussain, M. S. H. Zada, and B. Anderson, “Aerospace qualification services knowledge graph: A leap towards enhanced data management,” in *ISWC (Posters/Demos/Industry)*, 2023.
- [50] Anthropic, “Claude ai,” <https://claude.ai/>.



maintenance and data reliability.



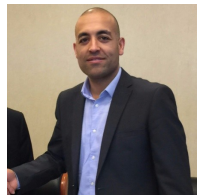
healthcare.



Raed Awill is an AI researcher specializing in knowledge graphs and aerospace AI applications. Currently pursuing an industrial PhD at the University of Derby and leading AI components in Innovate-UK funded aerospace projects at AddQual. His research focuses on autonomous verification systems for 4D spatiotemporal knowledge graphs, integrating multi-agent frameworks with adaptive intelligence. His contributions have been published in top-tier conferences, including ISWC and ESWC, and have driven advancements in AI-driven aerospace

Wajahat Ali Khan earned his MS in Information Technology from NUST, Pakistan (2010), and his PhD in Computer Engineering from Kyung Hee University, South Korea (2015). He is an Associate Professor at the School of Computing, University of Derby, UK, and previously served as an Assistant Professor at Kyung Hee University. In 2023, he received the Royal Society Industrial Research Fellowship. His research spans knowledge graphs, digital twins, ontology engineering, and decision support systems, with applications in aerospace and

Maqbool Hussain is a data scientist and researcher with over 15 years of experience in healthcare decision support systems. He has developed clinical decision support systems for tertiary hospitals in South Korea, Pakistan, and the USA. Currently, he serves as the Knowledge Base Supervisor for an Innovate UK-funded project on a Knowledge Graph for aerospace qualification services. His research, published in top-tier journals, focuses on Knowledge Graphs and digital twins for aerospace and healthcare applications.



Ben Anderson is a Chartered Engineer and founder of AddQual, specializing in aerospace and medical component manufacturing. He pioneered advanced manufacturing techniques and process qualification in turbine and precision engineering at Rolls-Royce and Sim-Cast. Under his leadership, AddQual earned AS9100 and NADCAP accreditation, serving major aerospace OEMs. His expertise also spans manufacturing process optimization, additive manufacturing, and quality assurance in safety-critical industries.



awards from prestigious conferences.

Ahsan Kazmi is a Senior Lecturer in Department of Computer Science and Creative Technologies, at the University of West of England, Bristol, UK. Prior to this, he worked as an Assistant Professor with the Institute of Information Security and Cyber Physical Systems, at Innopolis University, Russia. His research interests include applying analytical techniques of optimization, machine learning, and game theory to radio resource management for future networks. He also received the best KHU Thesis Award in engineering in 2017 and several best paper